

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Sistemas de Bases de Dados
1º Trabalho de Laboratório

Turma: 51D

Nome: João Ramos Número: 50730 Mail: A50730@alunos.isel.pt

Nome: Fábio Pestana Número: 50756 Mail: A50756@alunos.isel.pt

Nome: Rafael Dias Número: 50773 Mail: A50773@alunos.isel.pt

Professor: António Teófilo

Data: 18 de Novembro de 2024

Índice

Lista de Figuras 3
Introdução 4
1. Concepção 5
    1.1 - Modelo entidade associação. 5
    1.2 - Modelo relacional. 6
    1.3 - Restrições de Integridade Aplicacional (RIA). 10
2. Concretização 12
    2.1 Criação do modelo físico. 12
    2.2 Carregamento de dados de teste. 12
    2.3 Eliminação do modelo físico. 13
3. Conclusões 13
Bibliografia 14

Lista de Figuras

Figura 1 - Modelo entidade-associação 5

Introdução

Neste trabalho, desenvolvido para a unidade curricular de Sistemas de Bases de Dados, explorou-se os processos e metodologias envolvidas na criação de uma base de dados, desde a conceção de modelos conceituais, lógicos e físicos até a implementação utilizando a linguagem *Structured Query Language* (SQL).

O projeto proposto foca-se numa empresa de aluguer de veículos automóveis, que oferece serviços de aluguer de veículos. A empresa opera em múltiplos locais, atendendo tanto clientes individuais como empresas.

1. Concepção

1.1 - Modelo entidade associação.

Na figura seguinte, encontra-se o desenho do modelo entidade associação.

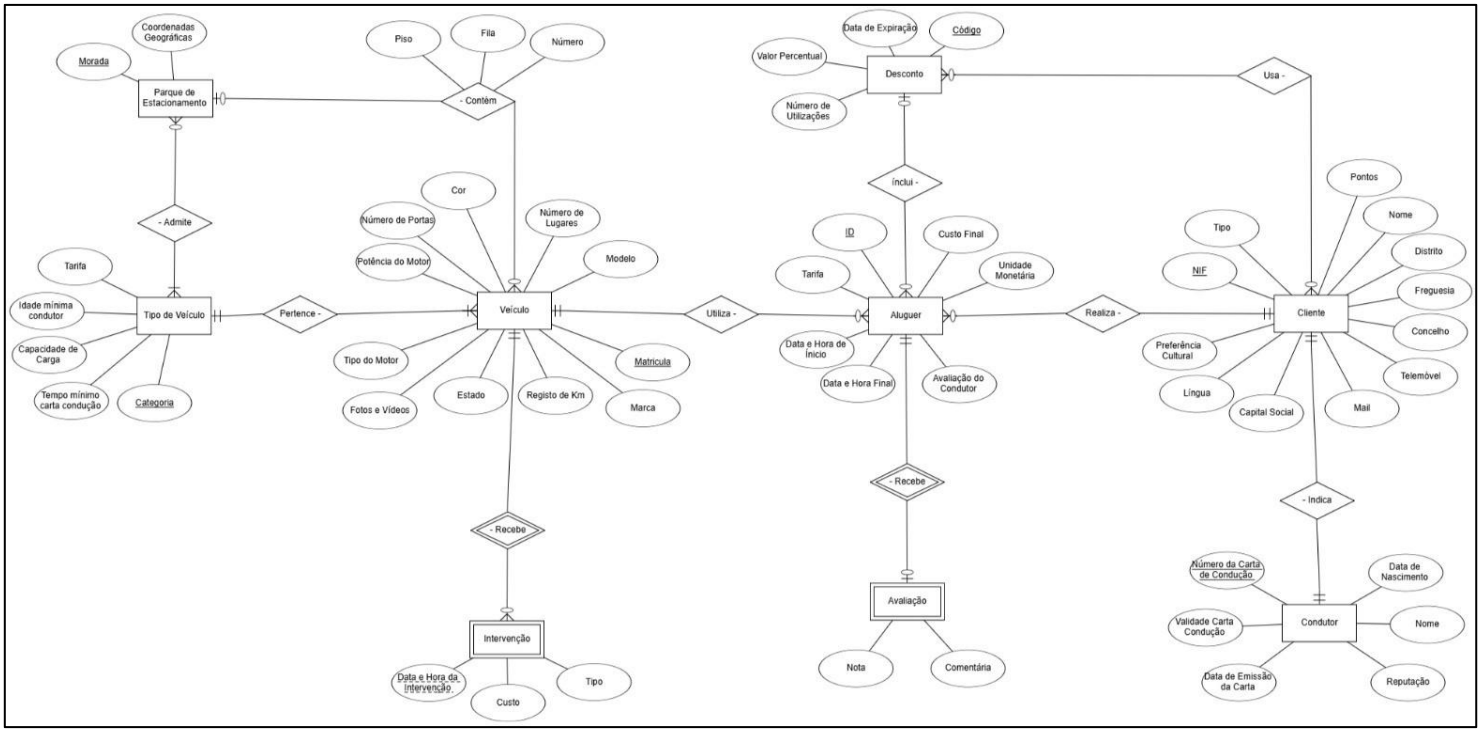


Figura 1 - Modelo entidade-associação

Na elaboração do modelo entidade associação, seguiu-se os seguintes pressupostos:

Parque de Estacionamento - A morada foi escolhida como chave primária pela possibilidade de existir dois ou mais parques muito próximos, o que pode acabar por causar problemas caso as coordenadas geográficas sejam escolhidas com pouca exatidão.

Tipo de Veículo - Foi pressuposto que a empresa pode comprar veículos pertencentes a um tipo que ainda nenhum dos seus parques aceite.

Intervenção - Foi pressuposto que os únicos tipos de intervenções que um veículo pode receber são reparações e manutenções.

Aluguer - Foi criado um atributo "ID" de modo a melhor identificar o aluguer. O atributo "Avaliação do Condutor" foi criado para que os condutores tenham a si atrelados uma reputação.

Cliente - O atributo "Pontos" foi criado de modo a fazer com que um cliente possa receber pontos ao realizar alugueres. Esses pontos podem ser gastos de modo a criar um desconto próprio de uso único.

Condutor - O atributo "Reputação" é uma média das avaliações que o condutor recebe pela empresa. As avaliações terão em conta o estado com que o veículo foi entregue à empresa depois de um aluguer.

1.2 - Modelo relacional.

O modelo relacional, apresenta a estrutura de dados para o sistema de uma empresa de aluguer de veículos, especificando as entidades, domínios dos atributos e os tipos de chaves (primárias, candidatas e estrangeiras). Cada entidade representa um aspeto fundamental do negócio, como veículos, clientes, parques de estacionamento, entre outros.

De seguida encontram-se as várias entidades criadas no modelo entidade-associação.

Parque de estacionamento (morada, coordenadas geográficas).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A morada é um `VARCHAR(150)`, ou seja, uma `string` com 150 caracteres.
- As coordenadas geográficas é um `VARCHAR(25)`, ou seja, uma `string` com 25 caracteres. Para assim registar a latitude e longitude com uma certa precisão.

Chave Primária: {morada}

Chaves Candidatas: {morada, coordenadas geográficas}

Tipo de veículo (categoria, tempo mínimo de carta de condução, capacidade de carga, idade mínima do condutor, tarifa).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A categoria é um `VARCHAR(30)`, que só aceita os seguintes valores: “familiar“, “comercial“, “motociclo“, “transporte de mercadorias“, “desportivo“.
- O tempo mínimo de carta de condução, a capacidade de carga, a idade mínima do condutor e a tarifa são números inteiros. O tempo mínimo da carta e idade mínima do condutor têm como unidade: anos. A capacidade de carga tem como unidade: quilogramas. A tarifa tem como unidade: euros/dia.

Chave Primária: {categoria}

Chaves Candidatas: {categoria, tarifa}

Veículo (matrícula, cor, marca, modelo, número de lugares, número de portas, potência do motor, tipo do motor, estado, fotos/vídeos, registo de km, categoria).

Domínio dos atributos:

- O único atributo que aceita `null` é o das fotos/vídeos, pois considera-se que não é obrigatório colocar fotos e vídeos do veículo.
- A matrícula é um `VARCHAR(10)`, ou seja, uma `string` com 10 caracteres.

- A cor e o modelo são um `VARCHAR(20)`, ou seja, uma `string` com 20 caracteres
- A marca é um `VARCHAR(25)`, ou seja, uma `string` com 25 caracteres.
- O número de lugares, número de portas e a potência do motor são números inteiros.
- O tipo do motor é um `VARCHAR(15)`, que só aceita os seguintes valores: "gasolina", "gasóleo", "GPL" e "elétrico, híbrido".
- O estado é um `VARCHAR(15)`, que só aceita os seguintes valores: "disponível", "alugado" e "manutenção".
- Fotos/vídeos é um `Blob`, ou seja, um valor binário de alta dimensão capaz de guardar informação sobre estes tipos de media.
- O registo de km é um número com casas decimais, representado por um `float`.

Chave Primária e Candidata: {matrícula}
Chave Estrangeira: {categoria}

Intervenção (data e hora da intervenção, custo, tipo, matrícula).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A data e hora da intervenção é um `DateTime`, ou seja, guarda informações sobre a data e hora da execução da intervenção no veículo.
- O custo é um número com casas decimais, representado por um `float`.
- O tipo é um `VARCHAR(20)`, que só aceita os seguintes valores: "reparação" e "manutenção".

Chave Candidatas: {data e hora da intervenção, matrícula}
Chave Estrangeira: {matrícula}

Aluguer (ID, tarifa, unidade monetária, data e hora de início, data e hora final, custo final, avaliação do condutor, matrícula, NIF).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- O ID é um `VARCHAR(10)`, ou seja, uma `string` com 10 caracteres.
- A tarifa é um número inteiro.
- A unidade monetária é um `VARCHAR(10)`, que só aceita os valores de uma lista de unidades monetárias. Lista {'EUR', 'USD', 'GBP', 'BRL', 'JPY', 'CHF','CNY}
- A data e hora de início e data e hora final são ambos um `DateTime`, para assim conseguirem guardar informações sobre a data e a hora.
- O custo final é um número com casas decimais, representado por um `float`.
- A avaliação do condutor é um inteiro, com valores entre 1 e 5, sendo 5 a indicação que o veículo foi entregue nas mesmas condições do seu levantamento. O valor 1 indica que o veículo foi entregue

com danos ou em péssimas condições.

Chave Primária: {ID}

Chave Candidatas: {ID, data e hora de início, data e hora final}

Chave Estrangeira: {matrícula} - Veículo; {NIF} - Cliente

Desconto (código, número de utilizações, data de expiração, valor percentual).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- O código é um `VARCHAR(12)`, ou seja, uma `string` com 12 caracteres.
- O número de utilizações é um número inteiro. Caso o desconto tenha uso ilimitado durante um período, o valor guardado na base de dados deve ser -1.
- A data de expiração é um `Date`, ou seja, guarda informações sobre a data.
- O valor percentual é um número inteiro.

Chave Primária e Canditada: {código}

Avaliação (ID, nota, comentário).

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- A nota é um *enumerator* (`Enum`), com valores que representam a avaliação feita pelo cliente ao escolher entre três valores possíveis: “Não vou voltar”, “Gostei” e “Adorei”.
- O comentário é um `VARCHAR(500)`, ou seja, uma `string` com 500 caracteres, optou-se por um `VARCHAR` em fez de um `TEXT`, para assim limitar o tamanho do texto.

Chave Candidatas: {comentário, ID}

Chave Estrangeira: {ID}

Cliente (NIF, tipo, nome, distrito, concelho, freguesia, mail, telemóvel, capital social, língua, preferência cultural, pontos).

Domínio dos atributos:

- Caso a morada do cliente não seja portuguesa, os atributos Distrito, Concelho e Freguesia podem ter valores `null`.
- Se o cliente for uma pessoa individual, o atributo capital social pode ter valor de `null`.
- A preferência cultural pode ter valor `null`, os restantes atributos não podem ter valores a `null`.
- O NIF é um `VARCHAR(12)`, ou seja, uma `string` com 12 caracteres.
- O tipo é um `VARCHAR(10)`, que só aceita os valores: "empresa" e "pessoa".
- O nome, concelho e a freguesia são `VARCHAR(35)`, ou seja, `strings` com 35 caracteres.
- O distrito e a língua são `VARCHAR(20)`, ou seja, `strings` com 20 caracteres.

- O mail é um `VARCHAR(70)`, ou seja, uma `string` com 70 caracteres.
- O telemóvel é um `VARCHAR(15)`, ou seja, uma `string` com 15 caracteres.
- O capital Social é um número inteiro.
- A preferência cultural é um `VARCHAR(50)`, ou seja, uma `string` com 50 caracteres.
- Os pontos é um número inteiro, que indica os pontos acumulados do cliente (podem ser usados em descontos).

Chave Primária: {NIF}

Chave Candidatas: {NIF, capital social, mail, telemóvel}

De seguida encontram-se as várias associações entre entidades do modelo entidade-associação.

Parque de estacionamento_Veículo (morada, piso, fila, número, matrícula).

Cada parque de estacionamento pode conter vários veículos, mas também pode não conter nenhum (caso não haja veículos estacionados no momento). Cada veículo está associado a um parque de estacionamento, mas essa associação não é obrigatória, pois o veículo pode estar alugado e, portanto, fora de qualquer parque. Caso esteja no parque, ele poderá ser encontrado no piso x, fila y, no número z.

Chave Candidata: {matrícula}

Chaves estrangeiras: {matrícula} e {morada}

Parque de estacionamento_Tipo de veículo (morada, categoria).

Cada parque de estacionamento tem de admitir pelo menos um tipo de veículos. Cada tipo de veículo pode ser admitido em vários parques, mas também pode não estar associado a nenhum parque (a empresa pode obter um tipo de veículo novo sendo que este ainda não é aceite por nenhum parque).

Chave Candidata: {morada, categoria}

Chaves estrangeiras: {morada} e {categoria}

Desconto _Aluguer (ID, código).

Um aluguer pode utilizar um desconto, mas como não é necessário ter um desconto para realizar um aluguer, este tem de ser opcional. No entanto, um desconto pode ser utilizado em vários diferentes alugueres (caso de descontos para vários clientes, como descontos de Natal).

Chave Candidata: {ID}

Chaves estrangeiras: {ID} e {código}

Cliente_Condutor (NIF, número da carta de condução, nome, data de emissão da carta, data de nascimento, validade da carta de condução, reputação).

Cada cliente deve obrigatoriamente indicar um condutor para o aluguer, que pode ser ele próprio ou outra pessoa. A indicação do condutor é essencial, pois o sistema precisa de registar quem será o responsável pela condução do veículo.

Domínio dos atributos:

- Nenhum dos atributos pode ser `null`.
- O número da carta de condução é um VARCHAR(12), ou seja, uma string com 12 caracteres.
- O nome é um `VARCHAR(35)`, ou seja, uma `string` com 35 caracteres.
- A data de emissão da carta, a data de nascimento e a validade da carta de condução são do tipo `Date`, ou seja, guardam informações sobre as datas.
- A reputação é um número com casas decimais, representado por um `float`.

Chave Candidatas: {NIF, número da carta de condução}

Chaves estrangeiras: {NIF}, como o cliente tem mais associações com outras entidades, ele é uma entidade própria logo a chave estrangeira é a primária do cliente.

Cliente_Desconto (NIF, código).

Cada cliente pode utilizar vários descontos ao longo do tempo, mas também pode não utilizar nenhum. Por outro lado, cada desconto pode ser utilizado por vários clientes diferentes, pois as promoções geralmente estão disponíveis para vários utilizadores. Como um cliente pode optar por não usar nenhum desconto ou um desconto pode não ser utilizado a associação é opcional para ambas as entidades.

Chave Candidata: {NIF, código}

Chaves estrangeiras: {NIF} e {código}

1.3 - Restrições de Integridade Aplicacional (RIA).

As Restrições de Integridade Aplicacional (RIA) representam as regras e procedimentos necessários que não conseguem ser apresentados nos modelos anteriores, logo estas regras são implementadas na aplicação. Abaixo encontram-se as restrições usadas no projeto.

Decremento do Número de Utilizações de um Desconto: Cada vez que um cliente utiliza um desconto disponível, é necessário decrementar o número de utilizações restantes desse desconto, evitando assim o uso excessivo do desconto e garantindo que o número de utilizações seja atualizado na base de dados. Caso o número de utilizações seja -1, isto é, uso ilimitado, o valor não deve ser decrementado.

Cálculo do Custo Final do Aluguer: O custo final do aluguer é determinado com base nas tarifas aplicáveis, considerando a duração do aluguer e o tipo de veículo. Caso o cliente tenha direito a um desconto válido, este é aplicado sobre o custo calculado. O custo final também deve ser apresentado segundo a unidade monetária escolhida.

Adição de Fotos e Vídeos (Conversão): A adição de fotos e vídeos à base de dados será pela aplicação, visto que tanto fotos quanto vídeos precisam ser previamente convertidos para binário ou Base64.

Mudança da Nota (Avaliação) para um Enum: A avaliação é feita pelo cliente ao escolher entre três campos: "Não vou voltar", "Gostei", "Adorei". A aplicação terá de converter essa escolha para um `Enum` a ser guardado na base de dados.

Geração de um ID para o Aluguer: Para cada novo aluguer, um identificador único (ID) deve ser gerado automaticamente. Este ID é essencial para identificar cada aluguer e avaliação no sistema.

Conversão da Data de Nascimento do Condutor para Idade: A data de nascimento do condutor deve ser convertida em idade no momento do aluguer, para ser comparada com a idade mínima exigida para conduzir o tipo de veículo selecionado.

Data e hora final do aluguer não deve ser superior à data de validade da carta do condutor: A data e hora final do aluguer, deve ser convertida numa data normal (AAAA-MM-DD) para assim ser comparada com a data de validade da carta do condutor, assim impedindo que o condutor alugue o veículo até essa data.

Um aluguer para ser válido tem de estar no intervalo (1 hora a 14 dias): A diferença entre a data inicial e final do aluguer deve encontrar-se no intervalo mencionado.

Cálculo da reputação do condutor: A reputação do condutor será calculada pela média de avaliações recebidas pelo condutor em cada um dos alugueres que ele participou.

Criação de um desconto próprio com pontos acumulados pelo cliente: O cliente, ao acumular pontos, pode gastá-los para criar um desconto próprio de uso único, em que o código será gerado utilizando um algoritmo e o NIF do cliente.

Colocar dados dos Clientes a `null`: Caso um cliente da empresa seja uma pessoa, esta não necessita de indicar a sua capital social (dado fica a `null`). Caso o cliente não tiver uma morada portuguesa, as informações sobre distrito, concelho e freguesia não serão preenchidas, isto é, serão colocadas a `null`.

2. Concretização

2.1 Criação do modelo físico.

Logo após a conceção do modelo entidade associação, modelo relacional e a definição das restrições de integridade aplicacional, foi iniciada a concretização da base de dados a partir de um *script* SQL DDL.

Primeiramente foi criada uma base de dados chamada **SBD_TP1** utilizando o comando `CREATE DATABASE SBD_TP1`, esta base de dados irá conter as tabelas das entidades e associações presentes nos modelos previamente apresentados. Usando a base de dados SBD_TP1 (`USE SBD_TP1`) foram criadas várias tabelas com o comando `CREATE TABLE` para cada uma das entidades e associações entre elas. Cada tabela contem diversas colunas que correspondem aos atributos pertencentes às entidades, indicando o seu tipo (Ex: VARCHAR(x), INT, BLOB, FLOAT), a possibilidade do atributo não conter nenhum valor (`Null`) e se o atributo é uma chave primária (*Primary Key*).

Caso um atributo possa apenas fazer parte de uma lista de valores podemos usar a função `CHECK IN`, em que "CHECK" seleciona o atributo a ser limitado e "IN" indica a lista de valores possíveis.

Após todas as tabelas terem sido criadas, foi indicado para cada uma as suas chaves estrangeiras (segundo o modelo relacional referido anteriormente). Isto considera-se uma boa prática, visto que todas as tabelas já estão criadas antes da adição das suas chaves estrangeiras, o que evita erros no que diz respeito à ordem em que as tabelas são criadas.

Por fim, criou-se quatro vistas, ou seja, criaram-se estruturas com o objetivo de simplificar o acesso a dados e facilitar consultas. As vistas criadas foram as seguintes:

- **AlugueresDetalhados**: Esta vista consolida informações detalhadas sobre os alugueres realizados. Ela reúne dados dos clientes, condutores e veículos, permitindo uma visualização completa de cada transação.
- **HistoricoDasIntervencoes**: Oferece o histórico das intervenções realizadas nos veículos, incluindo o tipo de intervenção (manutenção ou reparação), a data e o custo.
- **ResumoDosVeiculos**: Resume informações importantes sobre cada veículo, como o número total de alugueres realizados e a receita gerada.
- **InterpretacaoDosDescontos**: Esta vista traduz os descontos que não têm valores de utilizações, substituindo o seu valor de -1 pelo termo "Ilimitado". Isto facilita a interpretação por parte dos utilizadores e melhora a legibilidade dos dados.

2.2 Carregamento de dados de teste.

Seguidamente à criação do modelo físico foi criado um *script* SQL DML que tem como objetivo inserir dados nas tabelas criadas previamente, utilizando o comando `INSERT INTO VALUES`, em que "INSERT INTO" seleciona a tabela em que os dados serão inseridos e "VALUES" indica os valores que serão inseridos.

2.3 Eliminação do modelo físico.

Foi também criado um *script* SQL DDL para eliminar o modelo físico da base de dados. Para tal utilizamos a função `ALTER TABLE` que permite alterar uma tabela já criada, depois utilizamos a função `DROP CONSTRAINT` que permite a remoção das chaves estrangeiras das tabelas, com isto removemos as referências que uma tabela tem em relação com uma outra, impedindo assim erros posteriores na remoção de tabelas. De seguida, utilizamos a função `DROP TABLE IF EXISTS` que remove uma tabela da base de dados se esta existir. Por fim, utilizamos a função `DROP DATABASE IF EXISTS` que apaga a base de dados criada no primeiro ponto, se esta existir.

3. Conclusões

A elaboração deste trabalho permitiu consolidar conhecimentos sobre a modelação e implementação de bases de dados, destacando a importância de planear adequadamente o sistema desde a conceção até a concretização. O desenvolvimento de modelos conceptuais, seguido da implementação prática em SQL, demonstra a relevância de uma abordagem estruturada para atender aos requisitos de um sistema de informação robusto e eficiente.

Na fase de conceção, foram realizados os passos fundamentais para garantir a consistência e integridade do modelo de dados. A criação do modelo entidade-associação, fundamentada nos pressupostos assumidos, proporcionou uma visão clara das entidades e seus relacionamentos. A transformação desse modelo para o modelo relacional, com a definição de domínios, chaves e restrições de integridade, foi essencial para garantir a coerência da base de dados.

Bibliografia

Consulta:

Moodle: Trigo, P., Filipe, P., & Teófilo, A. (2024 - 2025). Sistemas de Bases de Dados